

회계가 바로 서야 경제가 바로 섭니다.

제09회 재무빅데이터분석사 자격시험

FDA 1급

Financial big-Data Analyst

■ 시험시간: 180분

■ 객관식배점: 문항별 배점 참조

■ 주관식배점: 문항별 배점 참조

- 회계처리기준 등을 적용하여 정답을 구하여야 하는 문제는 **시험시행 공고일 현재 시행 중인 법률·기준 등을 적용**하여 그 정답을 구하여야 합니다.
- 본 기출문제, 해설, 답안은 저작권법에 의해 보호받으므로 **무단복제와 무단전제를 금합니다.**

KICPA 한국공인회계사회

이론 객관식

01 다음과 같이 2차원 Array를 만들었다.

```
import numpy as np
arr = np.array([[2, 5, 99],
                [1, 7, 88],
                [2, 9, 77],
                [1, 3, 66],
                [2, 9, 55]])
```

상기 2차원 Array를 다음과 같이 나타내려고 한다.

```
[[ 1  7 88]
 [ 1  3 66]
 [ 2  9 77]
 [ 2  9 55]
 [ 2  5 99]]
```

상기와 같이 출력하기 위한 코드로 옳은 것을 고르시오. (3점)

㉠	i = np.lexsort((arr[:,1], arr[:,0])) print(arr[i])
㉡	i2 = np.argsort(-arr[:,1], kind='stable') arr1 = arr[i2] print(arr1)
㉢	i1 = np.argsort(arr[:,0], kind='stable') arr1 = arr[i1] i2 = np.argsort(-arr1[:,1], kind='stable') print(arr1[i2])
㉣	i = np.lexsort((-arr[:,1], arr[:,0])) print(arr[i])
㉤	print(arr[np.argsort(arr[:,0])][::-1])

① ㉠

② ㉡

③ ㉢

④ ㉣

⑤ ㉤

해설

- ㉣ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

정답 :

④

02 코드의 출력 결과가 다음과 같을 때 다음 보기 중 ㉠, ㉡, ㉢, ㉣에 모두 올바른 값이 들어간 것은? (3점)

```
import numpy as np
np.random.seed(7)
arr = np.random.randint(0, 100, size=(3, 4, 5))
filtered = arr[(arr >= ㉠) & (arr <= ㉡)]
print(filtered.reshape(㉢, ㉣))
```

```
[[47 25 23 57 23 42 39 48]
 [44 55 60 44 56 24 55 53]
 [34 56 38 48 31 44 23 35]]
```

㉠	㉠:20, ㉡:60, ㉢:3, ㉣:8
㉡	㉠:30, ㉡:70, ㉢:3, ㉣:8
㉢	㉠:20, ㉡:60, ㉢:2, ㉣:12
㉣	㉠:0, ㉡:50, ㉢:3, ㉣:8
㉤	㉠:40, ㉡:80, ㉢:3, ㉣:8

① ㉠

② ㉡

③ ㉢

④ ㉣

⑤ ㉤

해설

- ㉠ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

정답 :

①

03 다음은 전자상거래 주문 데이터프레임인 df_orders이다.

```
print(df_orders)
```

	OrderID	Product	Price	Quantity	Shipped
0	101	A	10.0	2	False
1	102	B	25.0	1	True
2	103	C	5.0	3	False
3	104	D	15.0	2	True
4	105	E	30.0	1	False

'Shipped' 컬럼이 True인 주문에 대해 'Total' 컬럼에 'Price'와 'Quantity'를 곱한 값을 계산하여 추가하고, 그렇지 않은 주문은 0을 넣으려고 한다. 다음은 이를 코드로 작성한 것이다.

```
import pandas as pd

data = {'OrderID': [101, 102, 103, 104, 105],
        'Product': ['A', 'B', 'C', 'D', 'E'],
        'Price': [10.0, 25.0, 5.0, 15.0, 30.0],
        'Quantity': [2, 1, 3, 2, 1],
        'Shipped': [False, True, False, True, False]}
df_orders = pd.DataFrame(data)
df_orders['Total'] = 0.0

for a in range(len(df_orders)):
    if df_orders.iloc[a, b] == True:
        df_orders.iloc[a, c] = df_orders.iloc[a, d] * df_orders.iloc[a, e]
```

㉠ ~ ㉡에 들어갈 내용으로 옳은 것을 고르시오. (3점)

㉠	㉠ : i, ㉡ : 4, ㉢ : 5, ㉣ : 2, ㉤ : 3
㉡	㉠ : i, ㉡ : 'Shipped', ㉢ : 'Total', ㉣ : 'Price', ㉤ : 'Quantity'
㉢	㉠ : i, ㉡ : 4, ㉢ : 5, ㉣ : 3, ㉤ : 'Price'
㉣	㉠ : i, ㉡ : 4, ㉢ : 5, ㉣ : 2, ㉤ : 0
㉤	㉠ : i, ㉡ : 5, ㉢ : 4, ㉣ : 2, ㉤ : 3

① ㉠

② ㉡

③ ㉢

④ ㉣

⑤ ㉤

해설

- ㉣ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

정답 :

①

04 다음은 상품 정보를 담은 데이터프레임 df_products이다.

```
print(df_products)
```

	상품명	카테고리	가격
0	TV	가전	1500000
1	T-shirt	의류	30000
2	노트북	가전	1200000
3	청바지	의류	80000
4	과자	식품	2000

'카테고리'가 '가전'인 상품은 15%, '의류'인 상품은 20% 할인된 최종 가격을 계산하여 '할인가' 컬럼에 추가하려고 한다. 그 외 카테고리는 할인을 적용하지 않는다. 다음은 이를 위한 코드이다.

```
import pandas as pd

def get_discount_price(a):
    category = a[b]
    price = a['가격']

    if category == '가전':
        return price * (1 - d)
    elif category == c:
        return price * (1 - d)
    else:
        return e

df_products['할인가'] = df_products.apply(get_discount_price, axis=f)
```

㉠ ~ ㉦에 들어갈 내용으로 옳은 것을 고르시오. (3점)

㉠	㉠ : product_row ㉡ : 0.2	㉢ : '카테고리' ㉣ : price	㉤ : '의류' ㉥ : 1
㉡	㉠ : product_row ㉡ : 0.2	㉢ : '가격' ㉣ : price	㉤ : '의류' ㉥ : 1
㉢	㉠ : product_row ㉡ : 0.15	㉢ : '카테고리' ㉣ : price	㉤ : '가전' ㉥ : 1
㉣	㉠ : product_row ㉡ : 20	㉢ : '카테고리' ㉣ : 0	㉤ : '의류' ㉥ : 1
㉤	㉠ : product_row ㉡ : 0.2	㉢ : '카테고리' ㉣ : price	㉤ : '의류' ㉥ : 0

- ① ㉠
④ ㉣

- ② ㉡
⑤ ㉤

- ③ ㉢

해설

- ㉗ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

정답 : ①

05 다음은 부서별, 직급별 직원들의 프로젝트 참여 횟수를 나타내는 데이터프레임 `df`이다.

```
print(df)
```

	부서	직급	프로젝트참여수
0	개발	사원	5
1	개발	대리	8
2	기획	과장	10
3	기획	사원	4
4	디자인	대리	7
5	개발	과장	12
6	기획	대리	9

판다스의 `pivot_table()` 함수를 사용하여 제품별/지역별 판매 데이터 분석을 수행하는 코드이다.

```
import pandas as pd

result = pd.pivot_table(df,
                        values=㉑,
                        index=㉒,
                        columns=㉓,
                        aggfunc=㉔,
                        margins=㉕,
                        fill_value=0).㉖

print(result)
```

위 코드를 실행했을 때, 아래와 같이 부서/직급별 프로젝트 참여 수 합계와 각 행/열의 총계('All')가 출력되게 하려고 한다. ('디자인' 부서의 '사원' 직급처럼 데이터가 없는 조합은 0으로 표시됨)

직급	부서	과장	대리	사원	All
0	개발	12	8	5	25
1	기획	10	9	4	23
2	디자인	0	7	0	7
3	All	22	24	9	55

㉑ ~ ㉖ 에 들어갈 내용으로 옳은 것으로 묶인 것을 고르시오. (3점)

㉠	a : '프로젝트참여수' d : np.mean	b : '부서' e : True	c : '직급' f : reset_index()
㉡	a : '프로젝트참여수' d : np.sum	b : '직급' e : True	c : '부서' f : reset_index
㉢	a : '프로젝트참여수' d : sum	b : '부서' e : True	c : '직급' f : reset_index()
㉣	a : '프로젝트참여수' d : 'sum'	b : '부서' e : False	c : '직급' f : reset_index
㉤	a : '프로젝트참여수' d : np.sum	b : '부서' e : False	c : '직급' f : reset_index()

① ㉠

② ㉡

③ ㉢

④ ㉣

⑤ ㉤

해설

- ㉢ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

 정답 :

③

06 다음은 직원 정보와 부서 정보를 담은 2개의 데이터프레임을 생성한 후 이를 조인하는 코드이다.

```
import pandas as pd

df_employees = pd.DataFrame({
    '직원ID': [101, 102, 103, 104],
    '이름': ['김철수', '이영희', '박민준', '최지우'],
    '부서코드': ['DEV', 'MKT', 'DEV', 'FIN']
})

df_departments = pd.DataFrame({
    '코드': ['DEV', 'MKT', 'HR'],
    '부서명': ['개발팀', '마케팅팀', '인사팀']
})

result = pd.merge(df_employees,
                  df_departments,
                  how=㉠,
                  left_on=㉡,
                  right_on=㉢)

print(result)
```

위 코드를 실행했을 때, 아래와 같이 모든 직원의 정보가 표시되지만, 부서 정보가 없는 '최지우'의 부서명은 NaN으로 출력되게 하려고 한다.

직원ID	이름	부서코드	코드	부서명	
0	101	김철수	DEV	DEV	개발팀
1	102	이영희	MKT	MKT	마케팅팀
2	103	박민준	DEV	DEV	개발팀
3	104	최지우	FIN	NaN	NaN

㉠ ~ ㉣ 에 들어갈 내용으로 옳은 것으로 묶인 것을 고르시오. (3점)

㉠	a : 'inner'	b : '부서코드'	c : '코드'
㉡	a : 'right'	b : '부서코드'	c : '코드'
㉢	a : 'left'	b : '부서코드'	c : '코드'
㉣	a : 'left'	b : '코드'	c : '부서코드'
㉤	a : 'outer'	b : '부서코드'	c : '코드'

① ㉠

② ㉡

③ ㉢

④ ㉣

⑤ ㉤

해설

- ㉣ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

정답 :

③

07 다음은 서버 접속/오류/방화벽 로그를 분석하여 ‘점검 대상 서버’를 선정하는 코드이다. 다음 세 조건을 모두 만족하는 서버를 점검 대상으로 판단한다.

- 누적 접속 시도 횟수 ≥ 100
- 오류 로그(error_log)에 등장
- 방화벽 차단 목록(firewall_block_log)에 없음

아래 코드에서 ㉠, ㉡, ㉢에 들어갈 알맞은 코드를 고르시오. (3점)

```
access_log = [
    {'server': 'WEB01', 'attempts': 50}, {'server': 'DB01', 'attempts': 120},
    {'server': 'WEB01', 'attempts': 60}, {'server': 'WAS01', 'attempts': 90},
    {'server': 'DB01', 'attempts': 30}
]
error_log = [ {'server': 'DB01'}, {'server': 'CACHE01'} ]
firewall_block_log = [ {'server': 'PROXY01'}, {'server': 'WEB01'} ]

check_required_servers = set()
server_stats = {}

for record in access_log:
    server_name = record['server']
    attempts = ㉠
    server_stats.setdefault(server_name, 0)
    server_stats[server_name] += attempts

servers_with_errors = { s['server'] for s in error_log }
servers_blocked = { s['server'] for s in firewall_block_log }

for server, total_attempts in ㉡:
    if total_attempts >= 100 and server in servers_with_errors:
        if server not in ㉢:
            check_required_servers.add(server)

print("점검 대상 서버:", sorted(check_required_servers))
점검 대상 서버: ['DB01']
```

㉠	㉠ : record['attempts']	㉡ : server_stats.items()	㉢ : servers_blocked
㉡	㉠ : record	㉡ : server_stats.keys()	㉢ : servers_blocked
㉢	㉠ : record['attempts']	㉡ : server_stats.items()	㉢ : servers_with_errors
㉣	㉠ : record['server']	㉡ : server_stats.values()	㉢ : servers_blocked
㉤	㉠ : record['attempts']	㉡ : server_stats	㉢ : servers_blocked

① 가

② 나

③ 다

④ 라

⑤ 마

해설

- ㉔ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

정답 : ①

08 다음 코드는 수입 품목별 관세 처리 상태를 정리하여 print한다. print 결과로 나오는 ㉑,㉒,㉓,㉔은 결과 그 자체가 아니라 그 위치에 나올 수 있는 것을 ㉑,㉒,㉓,㉔으로 표시한 것이다. 다음 설명 중 옳지 않은 것을 고르시오. (3점)

```
>>> 관세_사건 = { '문구류 수입': '관세 부과', '외교공관 물품': '관세 면제',
...   '개인 휴대품(한도 내)': '관세 면제', '고가 전자제품 해외직구': '관세 부과' }

>>> 우선순위 = {'관세 면제': 0, '관세 부과': 1}

... def is_duty(status: str) -> bool:
...     return status == '관세 부과'

>>> rows = [
...   (품목, 상태, '유발' if is_duty(상태) else '비적용')
...   for 품목, 상태 in 관세_사건.items()
... ]

>>> rows.sort(key=lambda t: (우선순위.get(t[1], 99), t[0]))

>>> for idx, (품목, 상태, 판정) in enumerate(rows, start=1):
...     print(f"{idx}. {품목}: {상태} → {판정}")
```

- ㉑
- ㉒
- ㉓
- ㉔

- ① `all(map(is_duty, 관세_사건.values()))` 는 모든 품목이 관세 부과 상태인지 여부를 True or False로 나타낸다.
- ② `any(s == '관세 면제' for s in 관세_사건.values())` 는 면제 품목 존재 여부를 불리언으로 나타낸다.
- ③ `sum(map(is_duty, 관세_사건.values()))` 는 관세 면제 품목 수를 센다.
- ④ ㉑은 개인 휴대품(한도 내): 관세 면제 → 비적용 이다.
- ⑤ ㉔은 고가 전자제품 해외직구: 관세 부과 → 유발 이다.

해설

- ③, `is_duty(status)`는 상태가 '관세 부과'일 때 `True(=1)`, 아니면 `False(=0)`를 반환한다. 따라서 `sum(...)`은 “관세 면제 품목 수”가 아닌 관세 부과 건수를 나타낸다.

 정답 : ③

09 다음은 텍스트에서 '키: 값' 형식의 데이터를 추출하는 코드이다. 키는 공백이 없는 문자열이고, 값은 따옴표(")로 둘러싸여 있으며, 값 내부에는 어떤 문자든 올 수 있다. 아래 코드의 ㉠에 들어갈 가장 적절한 정규표현식을 고르시오. (3점)

```
import re

text = 'name: "John Doe", age: "30", city: "New York"'

pattern = ㉠

results = dict(re.findall(pattern, text))

print(results)
{'name': 'John Doe', 'age': '30', 'city': 'New York'}
```

- ① `r'(\w+):\s*"(.+)"'`
- ② `r'(\S+):\s*"([^\"]*)"'`
- ③ `r'([a-z]+):\s*"(.*)"'`
- ④ `r'(\S+):\s*(".*")'`
- ⑤ `r'(\w+):\s*([\w\s]+)`

해설

- ②, `(\S+)`로 공백이 아닌 문자들의 연속을 캡처해 키를 잡아내고, 이어지는 `:\s*`로 콜론 뒤에 올 수 있는 임의의 공백을 허용한다.
- `"([^\"]*)"` 은 따옴표로 감싼 값을 매칭하여 가져오므로 `re.findall` 결과 `('name', 'John Doe')`, `('age', '30')`, `('city', 'New York')`이 되고 이를 딕셔너리로 변환하면 출력된 결과를 얻을 수 있다.

 정답 : ②

- 10 다음은 고객별 주문 데이터프레임 df이다. 각 고객의 첫 주문일로부터 며칠 후에 각 주문이 발생했는지를 계산하여 '첫주문후경과일' 컬럼을 추가하려고 한다. 아래 코드의 ㉠에 들어갈 가장 적절한 코드는 무엇인가? (3점)

```
import pandas as pd

# 데이터의 순서와 날짜의 최소값이 일치하지 않도록 수정
data = {'고객ID': ['A', 'B', 'A', 'C', 'B', 'A'],
        '주문일': ['2023-01-15', '2023-02-05', '2023-01-10',
                  '2023-03-01', '2023-02-20', '2023-02-01']}

df = pd.DataFrame(data)
df['주문일'] = pd.to_datetime(df['주문일'])

df['첫주문여부'] = ㉠

print(df.sort_values(['고객ID', '주문일'])) # 결과를 보기 쉽게 정렬하여 출력
```

	고객ID	주문일	첫주문여부
2	A	2023-01-10	True
0	A	2023-01-15	False
5	A	2023-02-01	False
1	B	2023-02-05	True
4	B	2023-02-20	False
3	C	2023-03-01	True

- ① df.groupby('고객ID')['주문일'].min()
- ② df['주문일'] == df.groupby('고객ID')['주문일'].transform('min')
- ③ df['주문일'] == df['주문일'].min()
- ④ df.rank(method='min', ascending=True) == 1
- ⑤ df.duplicated(subset=['고객ID'], keep='first')

해설

- ①, 그룹별 최소값 3개만 반환 → 행 길이가 불일치로 인해 원본 6행과 직접 비교·대입이 불가능하다.
- ②, 고객별 첫 주문일을 행 길이에 맞게 펼친 뒤 날짜와 비교하므로 옳은 보기이다.
- ③, 전체 데이터에서 가장 이른 날짜와 비교하므로 고객별 첫 주문 여부를 나타내지 않는다.
- ④, 열별(고객ID, 주문일)로 순위를 매겨 불리언 DataFrame(2열) 반환하므로 열 개수 불일치로 인한 오류가 발생한다.
- ⑤, duplicated(subset=['고객ID'], keep='first')는 행 순서를 기준으로 처음 등장한 고객만 False로 표시하므로 정답이 될 수 없다.

정답 : ②

주관식

11~14

당신은 (주)한공의 외부감사인이다. (주)한공은 ERP 프로그램을 사용하고 있으며 회계모듈에서 분개장을 입수하여 분석을 하려고 한다. 2010년 분개장을 csv로 받은 것이 분개장_1급_09회_1.csv 이며 이는 다음을 누르면 다운로드 받을 수 있다.

[분개장_1급_09회_1.csv](#)

분개장_1급_09회_1.csv 을 pandas의 데이터프레임으로 불러온다. Fraudit 테이블로 바로 import를 하면 레코드가 10000개로 제한되므로 pandas를 이용하여야 한다. 데이터프레임명은 '분개장'이라고 한다. 데이터프레임으로 불러온 후 데이터 타입을 살펴보면 다음과 같다.

컬럼명	데이터 타입	설명
전표일자	int64	전표일자
전표번호	int64	전표번호
증빙일자	int64	증빙일자
전표라인번호	int64	동일한 전표번호내의 라인번호
차대변지시자	object	차대변지시자(S:차변, H:대변)
현지통화금액	int64	현지통화금액
계정코드	int64	계정코드
계정과목	object	계정과목
거래처코드	object	거래처코드

동일한 전표일자에 동일한 전표번호는 하나의 전표세트를 의미한다. 즉, 아래의 네모 선 안에 있는 것과 같이 동일한 전표일자와 동일한 전표번호를 가지는 세트를 하나의 전표세트라고 한다. 아래의 네모 선 안에 있는 하나의 전표세트의 레코드수는 3개이다.

	전표일자	전표번호	전표라인번호	생성부서	생성자	계정코드	계정과목	차변금액	대변금액
0	2024-01-01	2024000110	1	영입부	70035	4020 내수상품매출액		0	9164
1	2024-01-01	2024000110	2	영입부	70035	2110 분기세금수금		0	916
2	2024-01-01	2024000110	3	영입부	70035	1270 내수외상매출금		10000	0
3	2024-01-01	2024000119	1	기회부	120188	1215 보험매출		3800000	0
4	2024-01-01	2024000119	2	기회부	120188	1270 내수외상매출금		0	3800000
5	2024-01-01	2024000216	1	영입부	60009	2087 거래보통금		17719396	0
6	2024-01-01	2024000216	2	영입부	60009	1270 내수외상매출금		0	17719396

11 차대변지시자가 S는 차변이고, H는 대변이다. 대변금액의 합계가 가장 큰 계정코드를 숫자만 입력하라. (5점)

🔒 정답 : 90003

해설

- (1) **New Script** 아이콘을 선택한다.
- (2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후 ▶를 누른다.

```
# 판다스 라이브러리 불러오기
import pandas as pd

# 파일 импорт
분개장 = pd.read_csv("분개장_1급_09회_1.csv")

# 차변금액, 대변금액 컬럼 삽입
분개장['차변금액'] = [ i[1] if i[0] == 'S' else 0 for i in zip(분개장['차대변지시자'],
분개장['현지통화금액'])]
분개장['대변금액'] = [ i[1] if i[0] == 'H' else 0 for i in zip(분개장['차대변지시자'],
분개장['현지통화금액'])]

# 요약 테이블 작성
gb_1 = 분개장.groupby(['계정코드', '계정과목'])

df_요약 = pd.DataFrame({
    '차변금액합계' : gb_1['차변금액'].sum(),
    '대변금액합계' : gb_1['대변금액'].sum(),
}).reset_index()
print(df_요약.sort_values(by = '대변금액합계'))
```

	계정코드	계정과목	차변금액합계	대변금액합계
106	40040	차이_24	5000	0
60	10139	건설중인자산	24837200	0
160	60093	운반비_2	89362072	0
55	10109	미착품	5760	0
162	60096	수출제비용_2	1953920	0
..	...	...	...	...
48	10102	반제품	2913363744	2195787586
47	10101	제품	2249712408	2236632780
4	10009	보통예금_1	0	2534514082
197	90001	CL_1	1337895460	4035048186
199	90003	CL_3	13042642810	26963974942

12 제품매출액_1(계정코드 40001)의 대변금액 합계가 가장 큰 월(month)의 숫자를 입력하라. 예를 들어, 9월이면 9만 입력하라. (5점)

정답 : 11

해설

- (1) New Script 아이콘을 선택한다.
- (2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후 ▶를 누른다.

```
# 판다스 라이브러리 불러오기
import pandas as pd

# 파일 임포트
분개장['월'] = pd.to_datetime(분개장['전표일자'], format='%Y%m%d').dt.month

# 제품매출액_1 발취
df_발취 = 분개장[분개장['계정코드'] == 40001]

# 요약 테이블 작성
gb_1 = df_발취.groupby('월')

df_요약 = pd.DataFrame({
    '차변금액합계' : gb_1['차변금액'].sum(),
    '대변금액합계' : gb_1['대변금액'].sum(),
}).reset_index()
print(df_요약.sort_values(by = '대변금액합계'))
```

	월	차변금액합계	대변금액합계
8	9	0	60146230
4	5	0	85485142
9	10	0	86927940
11	12	0	108982110
6	7	13530360	111688668
3	4	546000	126740582
7	8	10	128566900
1	2	1380000	147255602
0	1	13797080	157558520
2	3	0	169477514
5	6	42700	171450734
10	11	0	225144544

13 다음은 매출과 관련된 계정과목을 나타낸 것이다.

계정코드	계정과목	간주계정과목	계정코드	계정과목	간주계정과목
10003	당좌예금_1	매출상대	40001	제품매출액_1	매출
10004	당좌예금_2	매출상대	40002	제품매출액_2	매출
10005	당좌예금_3	매출상대	40004	상품매출액_1	매출
10006	당좌예금_4	매출상대	40005	상품매출액_2	매출
10009	보통예금_1	매출상대	40007	반제품매출액_1	매출
10011	보통예금_3	매출상대	40009	부산물매출액_1	매출
10012	보통예금_4	매출상대	40011	부재료매출액_1	매출
10013	보통예금_5	매출상대	40013	용역매출액_1	매출
10015	보통예금_7	매출상대	10017	보통예금_9	매출상대
10018	보통예금_10	매출상대	10021	보통예금_13	매출상대
10022	보통예금_14	매출상대	10023	보통예금_15	매출상대
10026	보통예금_18	매출상대	10027	보통예금_19	매출상대
10028	보통예금_20	매출상대	10032	보통예금_24	매출상대
10036	보통예금_28	매출상대	10037	보통예금_29	매출상대
10038	보통예금_30	매출상대	10039	보통예금_31	매출상대
10040	보통예금_32	매출상대	10044	보통예금_36	매출상대
10048	보통예금_40	매출상대	10050	보통예금_42	매출상대
10053	보통예금_45	매출상대	10054	보통예금_46	매출상대
10055	보통예금_47	매출상대	10057	보통예금_49	매출상대
10059	보통예금_51	매출상대	10060	보통예금_52	매출상대
10061	보통예금_53	매출상대	10063	보통예금_55	매출상대
10064	보통예금_56	매출상대	10065	보통예금_57	매출상대
10066	보통예금_58	매출상대	10067	보통예금_59	매출상대
10080	외상매출금_1	매출상대	10084	외상매출금_5	매출상대
10086	받을어음_1	매출상대	10087	받을어음_2	매출상대
10092	미수금_1	매출상대	20017	선수금_1	매출상대

정상적인 매출거래 전표세트의 정의는 다음과 같다.

- ① 간주계정과목이 매출이면서, 대변금액이 0을 초과하거나 차변금액이 0 미만인 경우가 포함된다.
- ② 간주계정과목이 매출상대 중 하나 이상이며, 차변금액이 0을 초과하거나 대변금액이 0 미만인 경우가 포함된다.

다음은 정상적인 매출거래 전표세트의 예이다.

전표일자	전표번호	계정코드	계정과목	차변금액	대변금액
20100101	1	XXXX	외상매출금_1	220,000	0
20100101	1	XXXX	선수부가세	0	20,000
20100101	1	XXXX	제품매출액_1	0	200,000

따라서, 정상적인 매출거래가 아닌 전표세트의 정의는 다음과 같다.

- ① 간주계정과목이 매출이면서, 대변금액이 0을 초과하거나 차변금액이 0 미만인 경우가 포함된다.
- ② 상대 간주계정과목이 매출상대가 아닌 항목들로만 나타난다.

다음은 정상적인 매출거래가 아닌 전표세트의 예이다.

전표일자	전표번호	계정코드	계정과목	차변금액	대변금액
20100101	2	XXXX	기계장치	330,000	0
20100101	2	XXXX	선수부가세	0	30,000
20100101	2	XXXX	제품매출액_1	0	300,000

정상적인 매출거래가 아닌 전표세트의 개수를 숫자로만 나타내라. 단, 천단위 콤마는 나타내지 않는다. (7점)

 정답 : 216

해설

- (1) **New Script** 아이콘을 선택한다.
- (2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후  를 누른다.

```
# 매출리스트
매출리스트 = [40001, 40002, 40004, 40005, 40007, 40009, 40011, 40013]
매출상대리스트 = [10003, 10004, 10005, 10006, 10009, 10011, 10012, 10013, 10015, 10017,
10018, 10021, 10022, 10023, 10026, 10027, 10028, 10032, 10036, 10037, 10038, 10039,
10040, 10044, 10048, 10050, 10053, 10054, 10055, 10057, 10059, 10060, 10061, 10063,
10064, 10065, 10066, 10067, 10080, 10084, 10086, 10087, 10092, 20017]
# 매출빈도 컬럼 삽입
분개장['매출빈도'] = (분개장['계정코드'].isin(매출리스트) &
((분개장['차변금액'] < 0) | (분개장['대변금액'] > 0))
).astype('int')
# 매출상대빈도 컬럼 삽입
분개장['매출상대빈도'] = (분개장['계정코드'].isin(매출상대리스트) &
((분개장['차변금액'] > 0) | (분개장['대변금액'] < 0))
).astype('int')
# groupby : 전표번호, 전표일자
gb_1 = 분개장.groupby(['전표일자', '전표번호'])
```

```

df_요약 = pd.DataFrame({
    '매출상대빈도' : gb_1['매출상대빈도'].sum(),
    '매출빈도' : gb_1['매출빈도'].sum(),
}).reset_index()
# 비정상 매출전표 발취
df_비정상 = df_요약[(df_요약['매출빈도'] > 0) & (df_요약['매출상대빈도'] == 0)]
# 개수 확인
print(len(df_비정상))

```

216

14 (㉞)한공의 매출과 관련된 계정코드와 계정과목은 다음과 같다.

계정코드	계정과목	간주계정과목	계정코드	계정과목	간주계정과목
40001	제품매출액_1	매출	40002	제품매출액_2	매출
40004	상품매출액_1	매출	40005	상품매출액_2	매출
40007	반제품매출액_1	매출	40009	부산물매출액_1	매출
40011	부재료매출액_1	매출	40013	용역매출액_1	매출

당신은 동일한 거래처에 대한 매출거래시 선수부가세가 0이 아닌 경우가 발생하는 경우와 {선수부가세가 0인 경우 또는 선수부가세가 나타나지 않는 경우}가 모두 발생하는 거래처가 있는지를 확인하려고 한다. 예를 들어, 다음과 같은 경우가 있는지를 알고 싶어한다.

전표일자	전표번호	계정코드	계정과목	차변금액	대변금액	거래처코드
20100101	1	XXXX	외상매출금_1	330,000	0	emp01
20100101	1	XXXX	선수부가세	0	30,000	emp01
20100101	1	XXXX	제품매출액_1	0	300,000	emp01
20100101	2	XXXX	외상매출금_1	100,000		emp01
20100101	2	XXXX	선수부가세	0	0	emp01
20100101	2	XXXX	제품매출액_1		100,000	emp01

전표일자	전표번호	계정코드	계정과목	차변금액	대변금액	거래처코드
20100101	1	XXXX	외상매출금_1	330,000	0	emp01
20100101	1	XXXX	선수부가세	0	30,000	emp01
20100101	1	XXXX	제품매출액_1	0	300,000	emp01
20100101	3	XXXX	외상매출금_1	200,000		emp01
20100101	3	XXXX	제품매출액_1		200,000	emp01

전표일자	전표번호	계정코드	계정과목	차변금액	대변금액	거래처코드
20100101	1	XXXX	외상매출금_1	330,000	0	emp01
20100101	1	XXXX	선수부가세	0	30,000	emp01
20100101	1	XXXX	제품매출액_1	0	300,000	emp01
20100101	2	XXXX	외상매출금_1	100,000		emp01
20100101	2	XXXX	선수부가세	0	0	emp01
20100101	2	XXXX	제품매출액_1		100,000	emp01
20100101	3	XXXX	외상매출금_1	200,000		emp01
20100101	3	XXXX	제품매출액_1		200,000	emp01

당신은 동일한 거래처에 대한 매출거래시 선수부가세가 0이 아닌 경우가 발생하는 경우와 {선수부가세가 0인 경우 또는 선수부가세가 나타나지 않는 경우}가 모두 발생하는 거래처가 몇 개인지 숫자만 입력하라. 단, 천단위 콤마는 나타내지 않는다. (8점)

 정답 : 4

해설

- (1) **New Script** 아이콘을 선택한다.
- (2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후 ▶를 누른다.

```
# 매출리스트
매출리스트 = [40001, 40002, 40004, 40005, 40007, 40009, 40011, 40013]
# 매출빈도 컬럼
분개장['매출빈도'] = ((분개장['계정코드'].isin(매출리스트)) &
                      ((분개장['차변금액'] < 0) | (분개장['대변금액'] > 0)))
                      .astype('int')
# 선수부가세빈도 컬럼
분개장['선수부가세빈도'] = ((분개장['계정코드'] == 20033) &
                             ((분개장['차변금액'] != 0) | (분개장['대변금액'] != 0)))
                             .astype('int')
# 선수부가세0빈도 컬럼
분개장['선수부가세0빈도'] = ((분개장['계정코드'] == 20033) &
                              ((분개장['차변금액'] == 0) & (분개장['대변금액'] == 0)))
                              .astype('int')
# 전표일자, 전표번호, 거래처번호로 그룹바이
gb_1 = 분개장.groupby(['전표일자', '전표번호', '거래처코드'])
# 요약 테이블
df_요약_1 = pd.DataFrame({
    '매출빈도' : gb_1['매출빈도'].sum(),
    '선수부가세빈도' : gb_1['선수부가세빈도'].sum(),
    '선수부가세0빈도' : gb_1['선수부가세0빈도'].sum(),
}).reset_index()
# 매출빈도 > 0 인 항목만 발체
df_매출_발체 = df_요약_1[df_요약_1['매출빈도'] > 0]
# 2차 요약
gb_2 = df_매출_발체.groupby('거래처코드')
df_요약_2 = pd.DataFrame({
    '매출빈도' : gb_2['매출빈도'].count(),
    '선수부가세빈도' : gb_2['선수부가세빈도'].sum(),
    '선수부가세0빈도' : gb_2['선수부가세0빈도'].sum(),
}).reset_index()

# 3차 발체
df_최종발체 = df_요약_2[ ((df_요약_2['선수부가세빈도'] != 0) & (df_요약_2['선수부가세0빈도'] != 0)) |
                        ((df_요약_2['선수부가세빈도'] + df_요약_2['선수부가세0빈도'] != df_요약_2['매출빈도']))]

print(len(df_최종발체))
```

- 15 당신은 (주)대한의 외부감사인이다. (주)대한은 ERP 프로그램을 사용하고 있으며 회계모듈에서 입수한 전표와 회사가 제시한 시산표를 입수하여 비교분석을 하려고 한다. 2015년 전표와 시산표는 각각 아래에서 다운로드 받을 수 있다.

[전표.csv](#)

[시산표.csv](#)

각각의 csv를 Fraudit 테이블로 변환한 후 데이터 타입을 살펴보면 다음과 같다.

[전표 테이블]

컬럼명	데이터 타입	설명
전기일자	String	전기일자
전표번호	String	전표번호
입력일자	String	입력일자
기표부서	String	전표를 입력한 부서
계정코드	Integer	계정코드
계정과목	String	계정과목
차변금액	Integer	차변금액
대변금액	Integer	대변금액

[시산표 테이블]

컬럼명	데이터 타입	설명
계정코드	Integer	계정코드
계정과목	String	계정과목
기초잔액	Integer	기초잔액
차변합계	Integer	차변합계
대변합계	Integer	대변합계
마감잔액	Integer	기초잔액 + 차변합계 - 대변합계

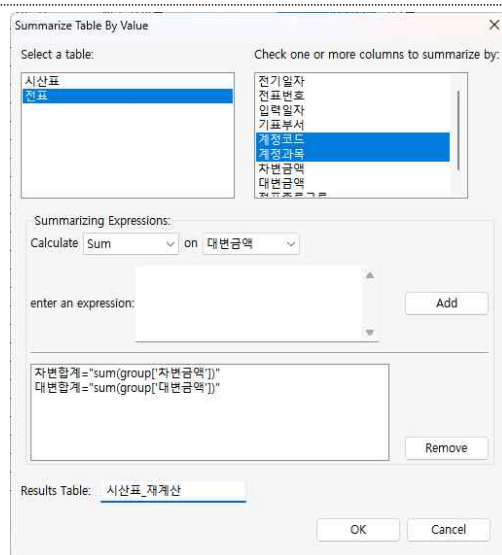
전표 테이블을 이용하여 계정코드(계정과목)별 차변합계와 대변합계를 구한 후 시산표의 기초잔액을 불러와 {기초잔액 + 차변합계 - 대변합계}를 다시 구한 후 시산표의 마감잔액과 비교하여 회사가 제시한 시산표의 정확성을 검증하려고 한다. 시산표의 마감잔액과 전표 테이블을 이용한 {기초잔액 + 차변합계 - 대변합계}가 일치하지 않는 계정코드의 개수를 숫자로만 나타내라. (5점)

 정답 : 2

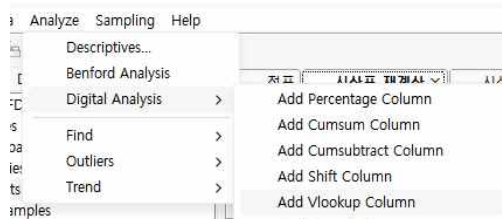
해설



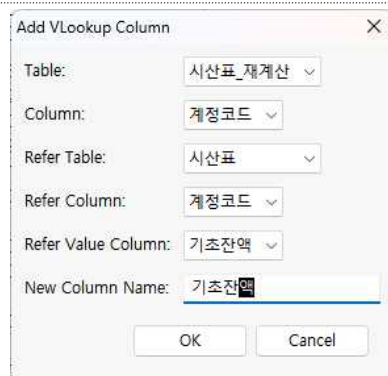
■ Data >> Summarize >> By Value 메뉴를 누른다.



- Select a table : 전표 선택
- Check one or more columns to summarize by : 계정코드, 계정과목 선택
- Summarizing Expressions
 - Calculate : Sum 선택
 - on : 차변금액 선택
 - Add 버튼을 누르고 차변합계 라고 입력
 - Calculate : Sum 선택
 - on : 대변금액 선택
 - Add 버튼을 누르고 대변합계 라고 입력
- Results Table : 시산표_재계산 입력
- OK 버튼을 누른다.



■ Analyze >> Digital Analysis >> Add Vlookup Column 메뉴를 누른다.



- Table : 시산표_재계산 선택
- Column : 계정코드 선택
- Refer Table : 시산표 선택
- Refer Column : 계정코드 선택
- Refer Value Column : 기초잔액 선택
- New Column Name : 기초잔액 선택
- OK 버튼을 누른다.

차변합계	대변합계
0	4240
17183200028	135678741
11992370	432138
0	1288

■ 오른쪽 클릭을 하고 Table properties 메뉴를 선택한다.

Table Properties: 시산표_재계산

Properties:

Name: 기말잔액

Type: Integer

Format: #

Calculation: 기초잔액 + 차변합계 - 대변합계

☐ Active Calculation (what's this?)

Save Cancel

■ 컬럼을 삽입한다.

- Name : **기말잔액**
- Type : **Integer**
- Calculation : 아래와 같이 입력한다.

기초잔액 + 차변합계 - 대변합계

■ Save 버튼을 누른다.

Analyze >> Digital Analysis >> Add Vlookup Column

시산표_재계산

시산표

계정코드

계정코드

마감잔액

마감잔액

■ Analyze >> Digital Analysis >> Add Vlookup Column 메뉴를 누른다.

Add Vlookup Column

Table: 시산표_재계산

Column: 계정코드

Refer Table: 시산표

Refer Column: 계정코드

Refer Value Column: 마감잔액

New Column Name: 마감잔액

OK Cancel

■ Table : **시산표_재계산** 선택

■ Column : **계정코드** 선택

■ Refer Table : **시산표** 선택

■ Refer Column : **계정코드** 선택

■ Refer Value Column : **마감잔액** 선택

■ New Column Name : **마감잔액** 선택

■ OK 버튼을 누른다.

Filter Expression: 마감잔액 != 기말잔액

계정코드 마감잔액 기초잔액 계정과목

■ Filter Expression에 다음과 같이 입력한다.

마감잔액 != 기말잔액

■ Set 버튼을 누른다.

계정코드	마감잔액	기초잔액	계정과목	차변합계	대변합계	기말잔액
1111008	175569669	319896902	신금	98027370	240294233	175569669
1121101	69046433	68930793	부채	25105100	24989461	69046432

■ 차이가 나는 계정과목은 2개임을 확인할 수 있다.

- 16 당신은 주요 기업의 재무제표 지표를 분석하고자 한다. 이를 위하여 아래의 사이트에서 csv를 다운로드 받는다. 아래 사이트에서 오른쪽 아래 CSV Export 버튼을 누르면 VALUESearch.csv가 다운로드 된다.

<http://fda.valuesearch.co.kr/>

CSV 파일을 Fraudit 테이블로 변환한 후 아래와 같이 데이터타입을 변경한다.

컬럼명	데이터타입	설명
종목코드	String	종목코드
업체코드	String	업체코드
종목명	String	종목명(기업명)
유동자산_2024	Decimal Number	2024년 유동자산 금액
비유동자산_2024	Decimal Number	2024년 비유동자산 금액
자산총계_2024	Decimal Number	2024년 자산총계
유동부채_2024	Decimal Number	2024년 유동부채 금액
비유동부채_2024	Decimal Number	2024년 비유동부채 금액
부채총계_2024	Decimal Number	2024년 부채총계
자본총계_2024	Decimal Number	2024년 자본총계
총수익_2024	Decimal Number	2024년 총수익
매출원가_2024	Decimal Number	2024년 매출원가
판매비와관리비_2024	Decimal Number	2024년 판매비와관리비
법인세비용차감전계속영업이익_2024	Decimal Number	2024년 법인세비용차감전계속영업이익
계속영업이익_2024	Decimal Number	2024년 계속영업이익
당기순이익_2024	Decimal Number	2024년 당기순이익
영업활동으로인한현금흐름_2024	Decimal Number	2024년 영업활동으로인한현금흐름
종업원수_2024	Integer	2024년 종업원수

매출총이익률은 (총수익 - 매출원가)/총수익으로 계산되며, 1인당 매출총이익률은 매출총이익률/종업원수로 계산된다. 2024년 1인당 매출총이익률이 가장 높은 기업의 종목명을 입력하라. 매출총이익률은 소수점 넷째짜리까지 반올림한다. (5점)

정답 : F&F홀딩스

- 17 (주)한공은 이제까지 향후 매출액을 결정론적 접근방식에 의하여 단일 평균값으로 추정하였다. 그러나 이 방법으로는 순이익 예측 모형에서의 불확실성을 제대로 반영하지 못한다는 컨설팅 의견을 받았다. (주)한공은 다음 회계연도의 주당순이익 (EPS : 순이익/발행주식수)을 110,000원 정도로 추정하고 있다. 이 추정이 틀렸을 경우 이로 인한 손해가 크기 때문에 재무이사(CFO)는 다음 회계연도의 순이익 (Net Income)과 주당순이익(EPS)에 대한 몬테카를로 시뮬레이션을 실행하려고 한다. 결정론적 접근방식에 의한 매출추정은 아래 매출추정.csv에 있다. 매출추정의 금액 단위는 원이다.

매출추정.csv

Filter Expression:	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
0	321300000	329400000	408900000	329500000	332400000	397200000	311100000	328700000	337400000	367900000	404600000	462700000

몬테카를로 시뮬레이션을 수행하기 위한 분포는 다음과 같다.

구분	분포	파라미터
월별 순매출	$N(\mu, \sigma)$	μ : 매출추정의 월별 순매출 σ : 월별 순매출 * 0.1
월별 매출원가율	$N(\mu, \sigma)$	μ : 0.60 σ : 0.01
월별 판관비율	$Tri(\min, \text{mode}, \max)$	$\min$: 0.065 mode : 0.074 $\max$: 0.081

구분	수치	비고
월별 감가상각	20,000,000원	고정금액
월별 이자비용	7,000,000원	고정금액
월별 법인세	월별 EBT × 20%	변동금액
발행주식수	100,000주	발행주식수

결과 변수의 산식은 다음과 같다.

구분	산식
매출총이익	순매출 - 매출원가
EBIT	매출총이익 - 판관비 - 감가상각비
EBT	EBIT - 이자비용
순이익	EBT - 법인세

(주)회사가 예측했던 주당순이익(EPS) 목표인 110,000원 이상 달성할 확률을 소수점 둘째 자리까지 반올림하여 구하라(예: 0.12345→0.12). 단, 시드는 np.random.seed(0)으로 고정시키며, 시뮬레이션 횟수는 100000으로 하고, 다음과 같은 라이브러리를 활용하라. (7점)

```
>>> import numpy as np
>>> import pandas as pd
>>> from pandas import *
>>> import mcerp
>>> from mcerp import *
>>> import matplotlib.pyplot as plt
```

 정답 : 0.80 또는 0.79

해설

- (1) New Script 아이콘을 선택한다.
- (2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후  를 누른다.

```
# 라이브러리 불러오기
import numpy as np
import pandas as pd
from pandas import *
import mcerp
from mcerp import *
import matplotlib.pyplot as plt
# 시뮬레이션 횟수 정하기
nR =100000
mcerp.npts = nR
# 시드 고정하기
np.random.seed(0)
# 입력변수의 파라미터 정하기
매출추정 = pd.read_csv('매출추정.csv')
순매출_평균 = [i for i in 매출추정.iloc[0][:]]
순매출_표준편차 = [i*0.1 for i in 순매출_평균]
# 입력변수의 분포 및 고정금액 설정하기
순매출_입력분포_리스트=[N(i[0],i[1]) for i in zip(순매출_평균, 순매출_표준편차)]
매출원가율_입력분포 = N(0.60, 0.01)
관관비율_입력분포 = Tri(0.065, 0.074, 0.081)
감가상각비 = 20_000_000
세율 = 0.20
이자비용 = 7_000_000
발행주식수 = 100_000
```

```

# 결과변수의 산식 설정하기
순이익 = 0
for i in 순매출_입력분포_리스트:
    매출총이익 = i - i*매출원가율_입력분포
    EBIT = 매출총이익 - i*판매비율_입력분포 - 감가상각비
    EBT = EBIT - 이자비용
    순이익 = 순이익 + EBT - EBT*세율
# EPS 구하기
EPS = 순이익/발행주식수
# 그래프 그리기
plt.figure("EPS")
EPS.plot()
plt.axvline(x =11000, color ='red')
plt.show()

```

```

# 임계치 이상 확률 구하기
print(EPS >= 11000)

```

```

0.79784

```

- 18 투자자 A는 주식의 과거 수익률 자료에 근거하여 5억원의 투자 금액으로 기대수익을 최대화 할 수 있는 포트폴리오를 구성하고자 한다. 포트폴리오의 내용은 다음과 같다.

구분	분포	수익률(평균)	표준편차
혼합형	정규분포	0.15	0.14
인덱스혼합형	정규분포	0.18	0.22
주식형	정규분포	0.17	0.30

구분	투자비율	최소 투자비율	최대 투자비율
혼합형		20%	80%
인덱스혼합형		30%	70%
주식형		15%	60%
합계	100%		

투자상품의 총투자 비율은 100% 이하이어야 하며, 각각의 투자비율의 범위는 상기 최소 투자비율 이상부터 최대 투자비율 이하이어야 한다. 한편 투자비율은 1% 단위로 변동된다. 기대수익을 최대화 할 수 있는 포트폴리오를 구성할 때 그 기대수익(평균)은 얼마인지 소수점 둘째 자리까지 반올림하여 구하라(예: 0.12345→0.12).

mcerp, geneticalgorithm 를 이용하고, 시뮬레이션을 10,000번 수행하고, 시드는 np.random.seed(0)으로 고정시킨다. 따라서 다음과 같이 먼저 라이브러리를 불러오고 설정을 한다. (8점)

```
# 관련 라이브러리를 불러온다.
import numpy as np
import mcerp
from mcerp import *
from geneticalgorithm import geneticalgorithm as ga
# 시드 고정하기
np.random.seed(0)
# 시뮬레이션 횟수 정하기
mcerp.npts = 10000
```

정답 : 0.17

해설

- (1) New Script 아이콘을 선택한다.
- (2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후 ▶를 누른다.

```
import numpy as np
import mcerp
from mcerp import *
from geneticalgorithm import geneticalgorithm as ga
```

```

# 시드 고정하기
np.random.seed(0) # seed를 0으로 고정시킨다.
# 시뮬레이션 횟수 정하기
mcerp.npts = 10000
# 입력변수의 분포 정하기
입력변수_1 = Normal(0.15, 0.14)._mcpts
입력변수_2 = Normal(0.18, 0.22)._mcpts
입력변수_3 = Normal(0.17, 0.30)._mcpts
# 목표함수 설정
def func(X) :
    # 전역변수 반영
    global 입력변수_1
    global 입력변수_2
    global 입력변수_3

    # 결정변수_1
    결정변수_1 = X[0]
    결정변수_2 = X[1]
    결정변수_3 = X[2]

    # 제약조건 위반시 페널티 반영
    페널티 = 0

    if 결정변수_1 + 결정변수_2 + 결정변수_3 > 100 :
        페널티 = -1000*(결정변수_1 + 결정변수_2 + 결정변수_3 -100) # ①

    # 결과변수 설정 #
    # 0.01을 곱하는 것은 ga() 함수의 파라미터 중 variable_type이 정수이므로 1% 변동 반영
    결과변수 = 입력변수_1*결정변수_1*0.01 + 입력변수_2*결정변수_2*0.01 \
        + 입력변수_3*결정변수_3*0.01 + 페널티

    return -np.mean(결과변수) # 최대화이므로 -를 반영한다.

var_bound=np.array([[20,80],
                    [30,70],
                    [15,60],])
model = ga(function = func,
            dimension = 3,
            variable_type = 'int', # 변동단위가 1%이므로 정수처리를 한다.
            variable_boundaries = var_bound)
model.run()

```

The best solution found:

[22. 63. 15.]

Objective function:

-0.17189852727039115

- 19 서울시의 업무추진비에 대한 벤포드 분석을 하고자 한다. 다음은 2022년 6월의 서울시 업무추진비 공개자료이다. 이는 아래에서 받을 수 있다.

업무추진비.csv

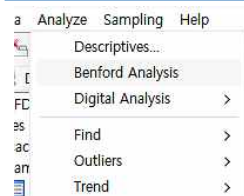
상기 파일을 **Fraudit** 테이블로 불러온 후의 테이블 구조는 다음과 같다.

컬럼명	데이터 타입	설명
title	String	업무명
url	String	url 주소
dept_nm_lvl_1	String	사용부서 레벨 1
dept_nm_lvl_2	String	사용부서 레벨 2
dept_nm_lvl_3	String	사용부서 레벨 3
exec_dt	String	사용일시
exec_loc	String	사용장소
exec_purpose	String	사용목적
target_nm	object	사용자 및 인원
payment_method	String	결제방법
exec_amount	Integer	사용금액

exec_amount의 앞의 2단위(예: 대변금액이 12345이면 12를 의미함)에 대해서 벤포드 분석을 하려고 한다. 앞의 2단위의 벤포드 평균이 **Expected**라고 가정할 경우 실제 발견된 비율(**Found Distribution**)의 Z값이 **6.0**을 초과하는 항목의 개수는 몇 개인지 숫자만 입력하라? 예를 들어 앞의 2단위가 **10, 11, 12, 13**이며 이들의 Z값이 **6.0**을 초과한다면 **4**를 입력한다. (6점)

답 : 6

해설



■ Analyze >> Benford Analysis 메뉴를 누른다.

Benford Analysis

Table: 업무추진비

Column to analyze: exec_amount

Number of significant digits: 2

Confidence Level: 95

Results name: 벤포드

☒ Append Benford's Expectation to the table

OK Cancel

- Table : 업무추진비 선택
- Column to analyze : exec_amount 선택
- Number of significant digits : 2 선택
- Confidence Level : 95% 선택 (반드시 95%를 선택할 필요는 없다)
- Results name : 벤포드 입력
- Append Benford's Expectation to the table : 체크
- OK 버튼을 누른다.

업무추진비 벤포드 벤포드_DistributionsReport x 벤포드_DistributionsChar

Filter Expression: Set

	First_2_Dig	Counts	Found	Expected	Z_score
0	50	181	.0278	.0086	16.7273
1	10	422	.0649	.0414	9.4799
2	60	102	.0157	.0072	8.0504
3	90	71	.0109	.0048	7.0493
4	80	73	.0112	.0054	6.3326
5	23	51	.0078	.0185	6.3260
6	26	55	.0085	.0164	4.9906
7	41	27	.0042	.0105	4.9430
8	84	62	.0095	.0051	4.8678

- Z값이 60을 초과하는 항목의 개수는 6개이다.

- 20 다음 random_data.csv에는 전화번호, 이름, 성별, 광역주소, 연봉이 랜덤으로 섞여 들어 있다. random_data.csv는 아래의 링크를 누르면 다운로드 받을 수 있다.

[random_data.csv](#)

random_data.csv의 레코드 수는 3000개이며 데이터의 1번째 ~ 3번째 레코드는 다음과 같다.

191,970,733원	이우진	남	광주광역시	063-628-1807
053-471-7330	남	전라남도	류광수	148,127,825원
인천광역시	장미정	₩152,213,860	여	051-290-9480

광역주소는 {'서울특별시', '부산광역시', '대구광역시', '인천광역시', '광주광역시', '대전광역시', '울산광역시', '세종특별자치시', '경기도', '강원도', '충청북도', '충청남도', '전라북도', '전라남도', '경상북도', '경상남도', '제주특별자치도'} 중 하나로 구성되어 있다.

『random_data.csv』를 데이터프레임으로 불러온 후 광역주소별 연봉합계 최고지역을 입력하라. 예를 들어 최고지역이 서울특별시이면 서울특별시라고 입력하라. (8점)

답 : 경상북도

21 다음은 풋옵션과 관련된 내용이다.

- | | | |
|---------------|----------------|------------------|
| • 아메리칸 풋옵션 | • 현재주가 : 200 | • 행사가격 : 210 |
| • 무위험이자율 : 6% | • 배당률 : 0% | • 만기 : 5년 |
| • 변동성 : 20% | • 스텝(노드) : 300 | • dt = 만기/스텝(노드) |

풋옵션 가격을 소수점 2째 자리까지 반올림하여 입력하라(예 : 0.12345→0.12).
단, 소수점 처리를 위해서 코드 상단에 np.set_printoptions(precision=2, suppress=True)를 입력해서 처리하라. (6점)

정답 : 22.72

해설

```
>>> import numpy as np
>>> np.set_printoptions(precision = 3, suppress = True)
# 기초변수 설정(한줄에 여러 변수를 설정하는 경우 세미콜론으로 구분함)
>>> 최초주가 = 200 ; 행사가격 = 210 ; 무위험이자율 = 0.06 ; 변동성 = 0.20
>>> 만기 = 5 ; 스텝 = 300 ; dt = 만기/스텝 ; 할인계수 = np.exp(-무위험이자율*dt)
# u와 d 계산
>>> U = np.exp(변동성 * np.sqrt(만기/스텝)) # u 계산
>>> D = np.exp(-변동성 * np.sqrt(만기/스텝)) # d 계산
# p와 (1-p) 계산
>>> P_U = (np.exp(무위험이자율*(만기/스텝))-D)/(U-D) # p
>>> P_D = 1 - P_U
# 추가트리 생성
>>> 추가_트리 = np.zeros([스텝 + 1, 스텝 + 1])
>>> for i in range(스텝 + 1):
...     for j in range(i + 1):
...         추가_트리[j, i] = 최초주가 * (U ** (i - j)) * (D ** j)
# 옵션트리 생성
>>> 옵션_트리 = np.zeros([스텝 + 1, 스텝 + 1])
# 마지막 스텝 : max((행사가격-추가), 0) 계산
>>> 옵션_트리[:,스텝] = np.maximum((행사가격 - 추가_트리[:,스텝]), 0)
>>> print(옵션_트리) # 소수점 2째자리 반올림
>>> for node in range(스텝-1,-1,-1) : #역순으로
...     for time in range(스텝-1,-1,-1) : #역순으로
...         if time >= node : #대각선을 기준으로 상단만 결과 나오게 함
...             옵션_트리[node, time] = np.maximum(행사가격-추가_트리[node,time],
...             (옵션_트리[node,time+1]*P_U+옵션_트리[node+1,time+1]*P_D)*할인계수)
>>> print(옵션_트리) # 소수점 2째자리까지 반올림
[[ 22.72  20.59  18.63 ...   0.    0.    0. ]
```

```

[ 0. 25.01 22.7 ... 0. 0. 0. ]
[ 0. 0. 27.49 ... 0. 0. 0. ]
...
[ 0. 0. 0. ... 209.91 209.91 209.9 ]
[ 0. 0. 0. ... 0. 209.91 209.91]
[ 0. 0. 0. ... 0. 0. 209.91]]

```